Machine Learning

Módulo 4

Support Vector Machine

Support Vector Machine

El siguiente gráfico representa dos grupos de puntos de clases distintas, que se intenta separar:

Para hacerlo, la solución más sencilla puede ser con una recta, sin embargo, existen infinitas rectas posibles que separan perfectamente los grupos de datos. Además, el mejor criterio para seleccionar una de esas rectas, es escoger una línea que capture el patrón general en los datos de entrenamiento, así cabe una buena posibilidad de que separe bien los datos de prueba ó nunca vistos.

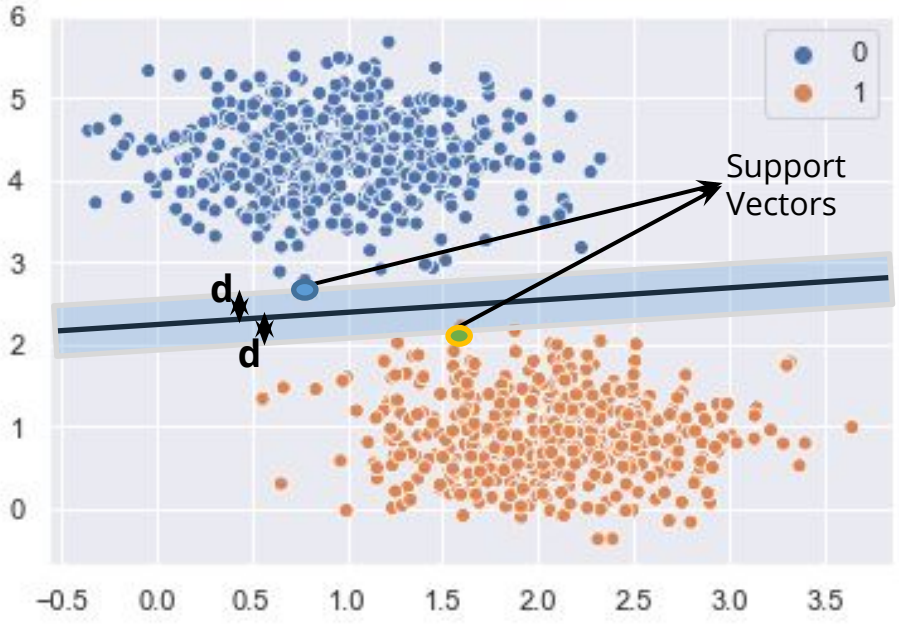


Support Vector Machine

Es necesario entonces:

- Buscar rectas que clasifiquen correctamente los datos de entrenamiento.
- Entre todas estas rectas, elegir la que tenga la mayor distancia **d**, a los puntos más cercanos a ella.

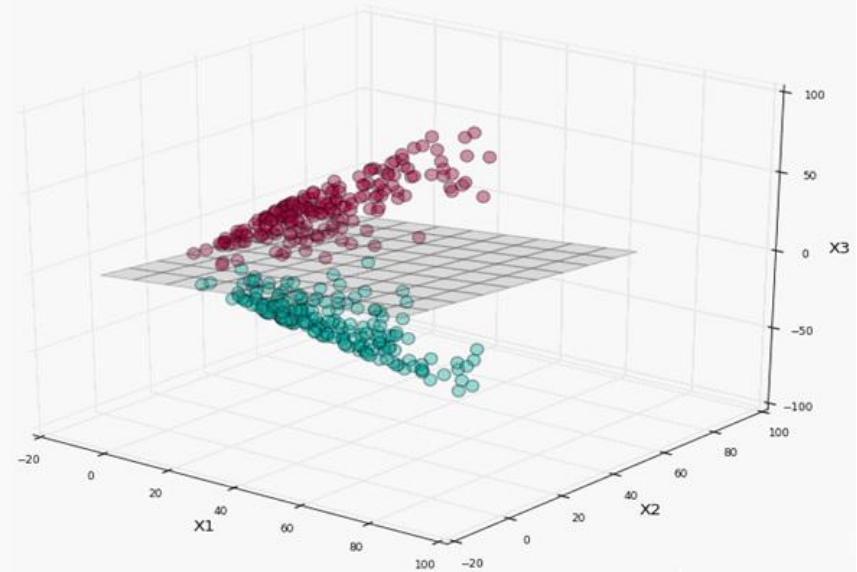
Los puntos más cercanos que identifican esta recta se conocen como **vectores de apoyo** (Support Vectors). Y la región que definen alrededor de la línea se conoce como el **Margen**.



Support Vector Machine

Los datos que pueden ser separados por una recta (o en general, un **hiperplano**) se conocen como *datos linealmente separables.* El hiperplano actúa como un clasificador lineal.

SVM es extensible para n dimensiones. En este caso, la recta de decisión se transformó en un **plano de decisión**, por tratarse de un problema de **3 dimensiones**.



Support Vector Machine

En la realidad, los datos NO suelen ser separables con una recta.

Aunque tampoco se quiere descartar por completo al clasificador lineal, ya que parece adecuado para el problema, excepto por algunos puntos erróneos.



Support Vector Machine

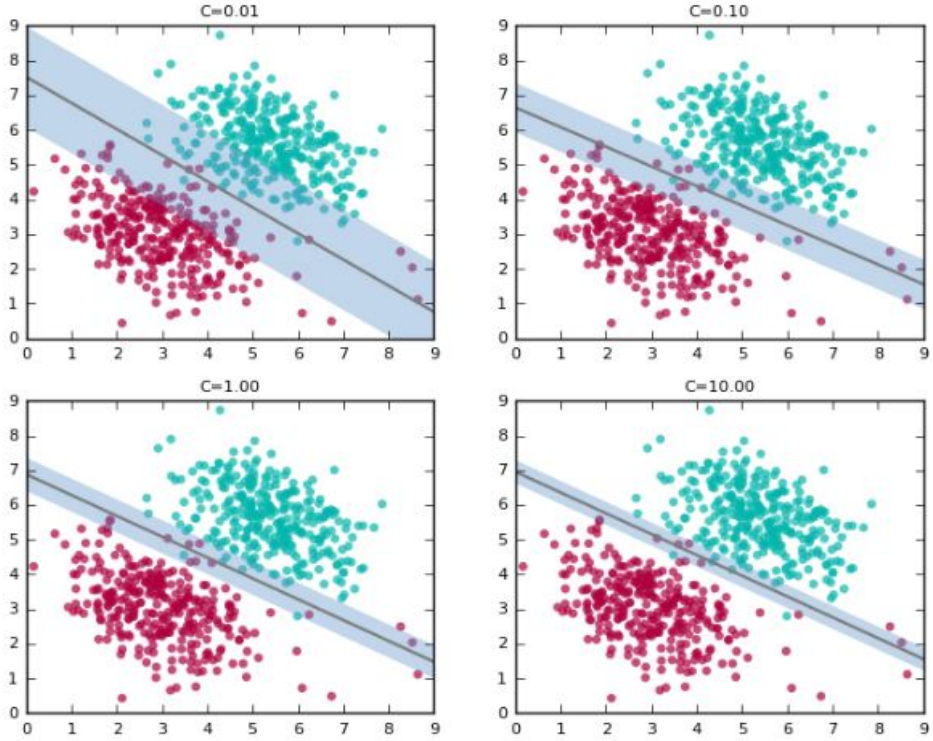
En SVM, se puede especificar cuántos errores estamos dispuestos a aceptar mediante un parámetro llamado C, lo que permite dictaminar la relación entre:

- Tener un amplio margen.
- Clasificar correctamente la mayor cantidad de puntos de entrenamiento, dar un valor más alto de C implica que se esperan menos errores en los datos de entrenamiento.



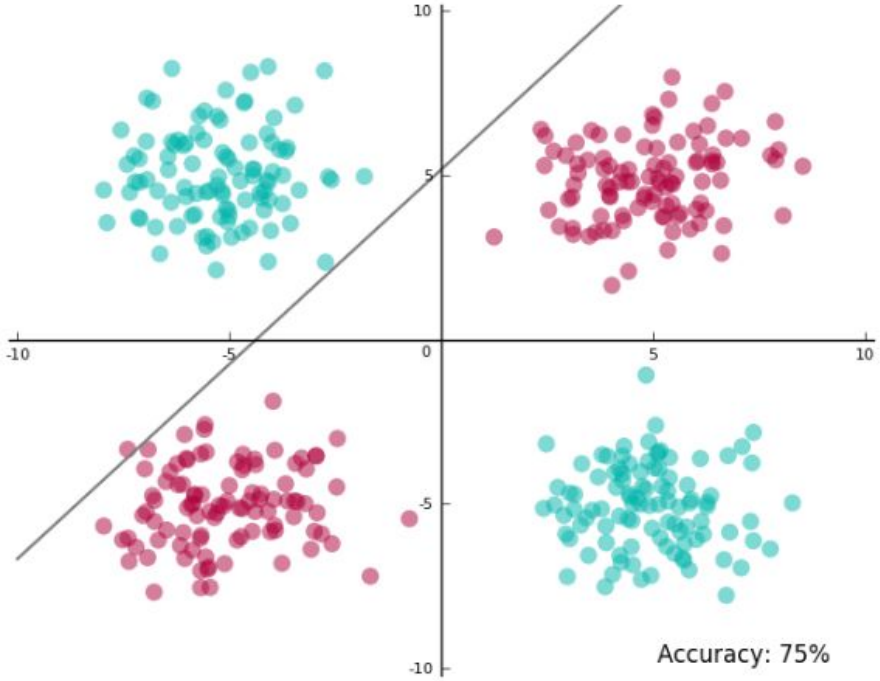
Support Vector Machine

- Se puede observar como la línea se inclina a medida que aumenta el valor de C.
- A valores altos, intenta acomodar las etiquetas de la mayoría de los puntos rojos presentes en la parte inferior derecha de los gráficos.
- Esto probablemente no es lo que queremos para los datos de prueba.
- El primer gráfico con C=0,01 parece captar mejor la tendencia general, aunque adolece de una menor precisión en los datos de entrenamiento en comparación con los valores más altos de C.



Support Vector Machine

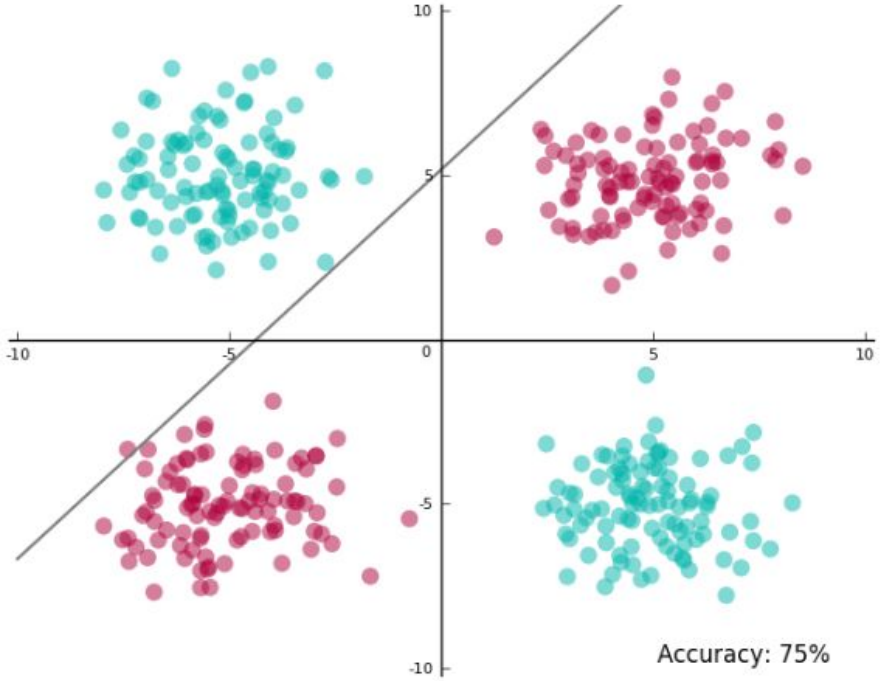
- No siempre es posible separar los datos con una recta, plano o hiperplano. En esos casos es posible **aumentar la dimensionalidad** de los datos, agregando nuevas dimensiones que permitan aplicar clasificadores lineales.
- En este caso, no podemos separar los datos con un recta, entonces lo que hacemos es aumentar de 2 a 3 dimensiones, mediante una proyección a un nuevo espacio de 3D.
- Lo mejor posible con la recta del ejemplo, es un 75% de precisión en los datos de entrenamiento



Support Vector Machine

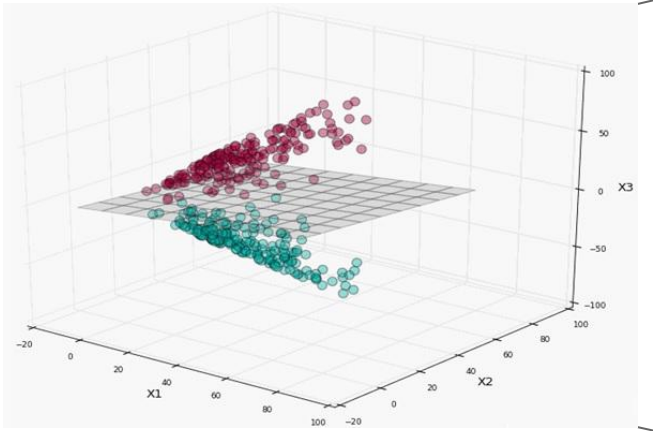
Esto se puede resolver entonces, agregando una dimensión más a los datos, por ejemplo de alguna de las siguientes formas:

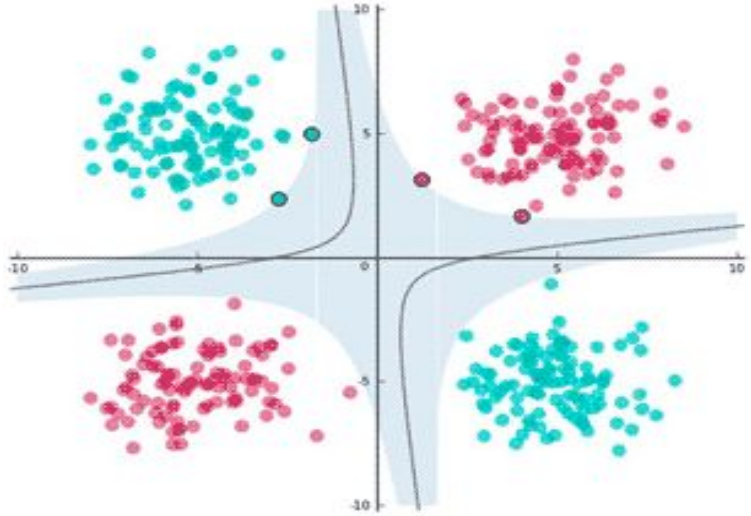
$$X_1 = x_1^2$$
$$X_2 = x_2^2$$
$$X_3 = \sqrt{2}x_1x_2$$



Support Vector Machine

Ahora, los datos proyectados con la nueva variable, sí pueden quedar separados:

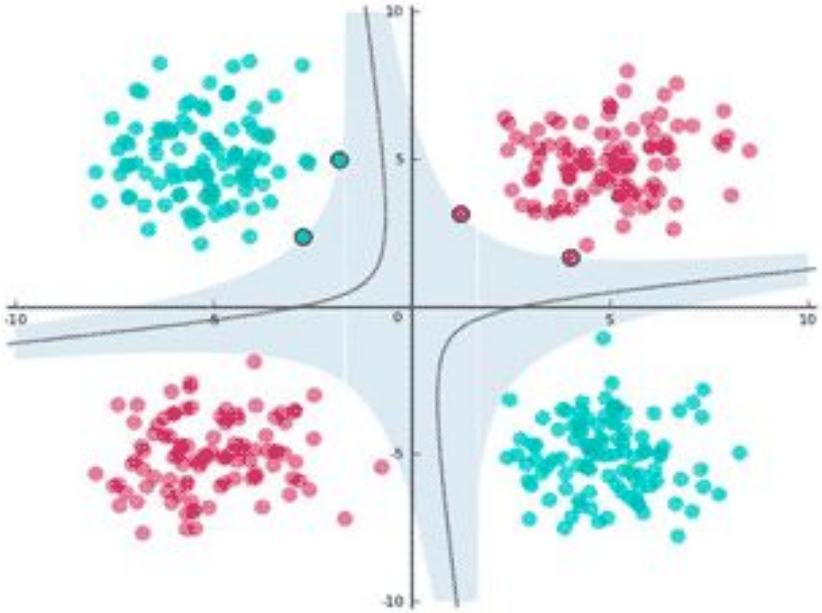




Para conocer el nuevo límite de separación, se mapea el hiperplano al espacio original.

Support Vector Machine

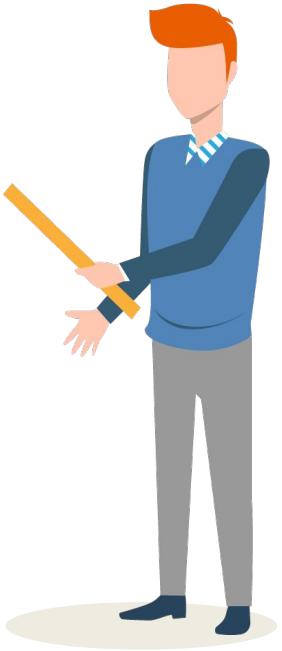
- La forma del límite de separación en el espacio original depende de la proyección.
- En el espacio proyectado, esto es siempre un hiperplano.
- Cuando se mapea de vuelta al espacio original, el límite de separación ya no es una recta. Esto también es cierto para los vectores de margen y de soporte.
- En cuanto a nuestra intuición visual, tienen sentido en el espacio proyectado.



Support Vector Machine

1. Para datos linealmente separables, SVM trabaja muy bien.
2. Para los datos que son casi linealmente separables, SVM puede funcionar bastante bien usando el valor correcto de C.
3. Para los datos que no son linealmente separables, podemos proyectar los datos a un espacio en el que sean perfectamente/casi linealmente separables, lo que reduce el problema a 1 ó 2.

Una gran parte de lo que hace que SVM sea universalmente aplicable es proyectar a dimensiones superiores. Y aquí es donde entran los **kernels**...



Support Vector Machine

Un Kernel es una forma de calcular el **producto punto** de dos vectores *x* e *y* en algún espacio de características (posiblemente de mayor dimensionalidad), por lo que las funciones del kernel a veces se denominan *"producto punto generalizado"*.

Supongamos que tenemos un mapeo $\phi : \mathbb{R}_n \rightarrow \mathbb{R}_{??}$ que trae nuestros vectores en $\mathbb{R}_n$ a algún espacio de características $\mathbb{R}_{??}$ Entonces, el producto punto de ***x*** e ***y*** en este espacio es $\phi(x)^T \phi(y)$

Un kernel es una función *K* que corresponde a este producto punto, es decir, $K(xy) = \phi(x)^T \phi(y)$

Ej.: Definimos un mapeo polinómico a un espacio 3D:

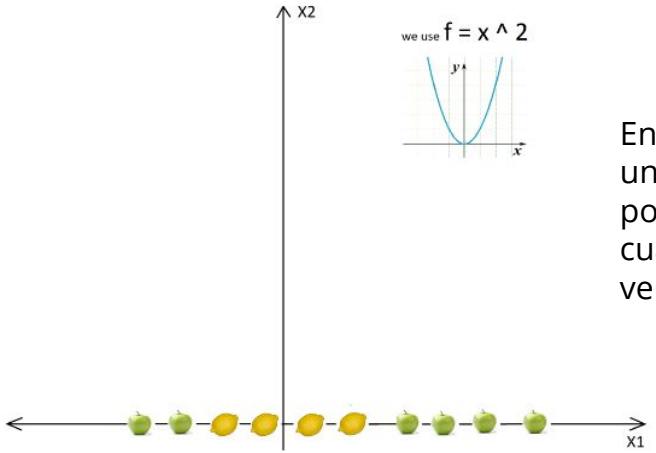
$$\phi(X) = \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}x_1 x_2 \end{bmatrix}$$

Entonces la función kernel asociada es: $K(xy) = \langle \phi(x), \phi(y) \rangle = \phi(x)^T \phi(y)$ $x_1^2 y_1^2 + x_2^2 y_2^2 + 2x_1 y_1 x_2 y_2$ =

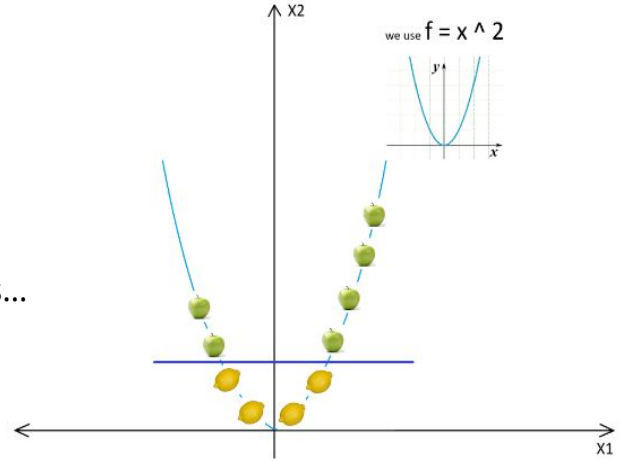
Support Vector Machine

Normalmente no se define una proyección específica para los datos. En su lugar, se selecciona entre kernels disponibles, ajustándolos en algunos casos, para encontrar el que mejor se adapte.

Por supuesto que nada nos impide definir nuestros propios kernels, o realizar la proyección nosotros mismos, pero en muchos casos no es necesario.



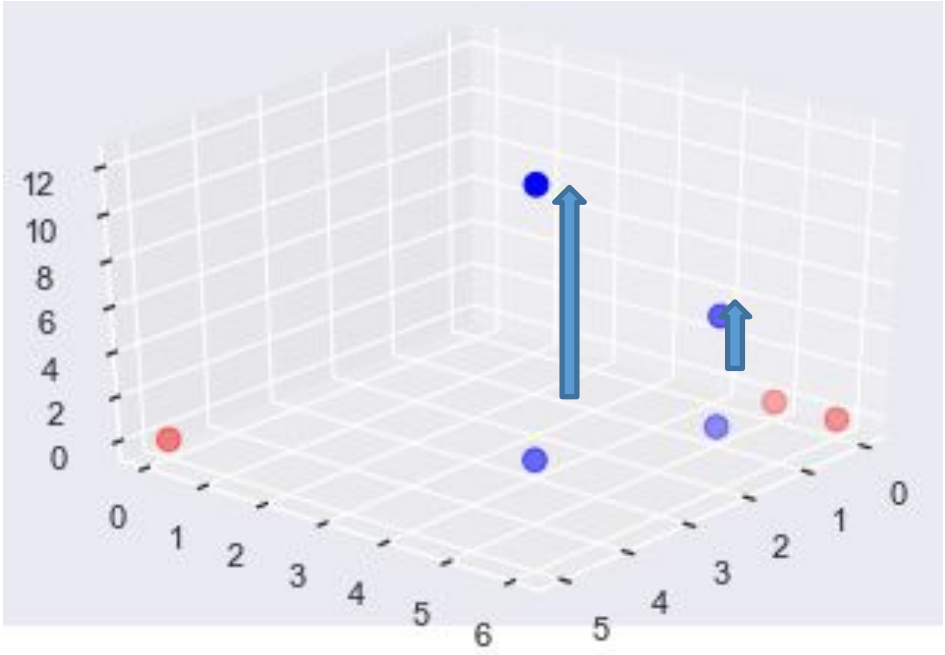
Entonces, por ejemplo, creamos una tercer variable que esté dada por la función X elevado al cuadrado, lo cual, como puede verse, logra separar las dos clases...



Support Vector Machine

En el siguiente ejemplo, tenemos que la función x*y es la que mejor separa los puntos rojos de los azules:

Punto	x+y	x*y	x^2+y^2
(0,6)	6	0	36
(0,5)	5	0	25
(1,5)	6	5	26
(3,4)	7	12	25
(5,0)	5	0	25



Support Vector Machine

Algunos Kernels disponibles:

- **Lineal**

$$k(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$$

- **Polinómico** (parámetros p y c)

$$k(\mathbf{x}_i, \mathbf{x}_j) = \left(\mathbf{x}_i^T \mathbf{x}_j + c\right)^p$$

- **Gaussiano** (parámetro σ^2)

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$$

- Podemos crear nuevos kernel mediante transformaciones
  1. $k_1(\mathbf{x}, \mathbf{y}) + k_2(\mathbf{x}, \mathbf{y})$
  2. $k_1(\mathbf{x}, \mathbf{y}) k_2(\mathbf{x}, \mathbf{y})$
  3. $\exp(k_1(\mathbf{x}, \mathbf{y}))$

Support Vector Machine

Implementación con Scikit-Learn:

```
from sklearn.svm import SVC
```

```
classifier = SVC(kernel='sigmoid', gamma='auto')
```

```
classifier.fit(X1, y1)
```

```
SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,
    decision_function_shape='ovr', degree=3, gamma='auto', kernel='sigmoid',
    max_iter=-1, probability=False, random_state=None, shrinking=True,
    tol=0.001, verbose=False)
```

```
classifier.predict([[-1,2]])
```

```
array([0])
```

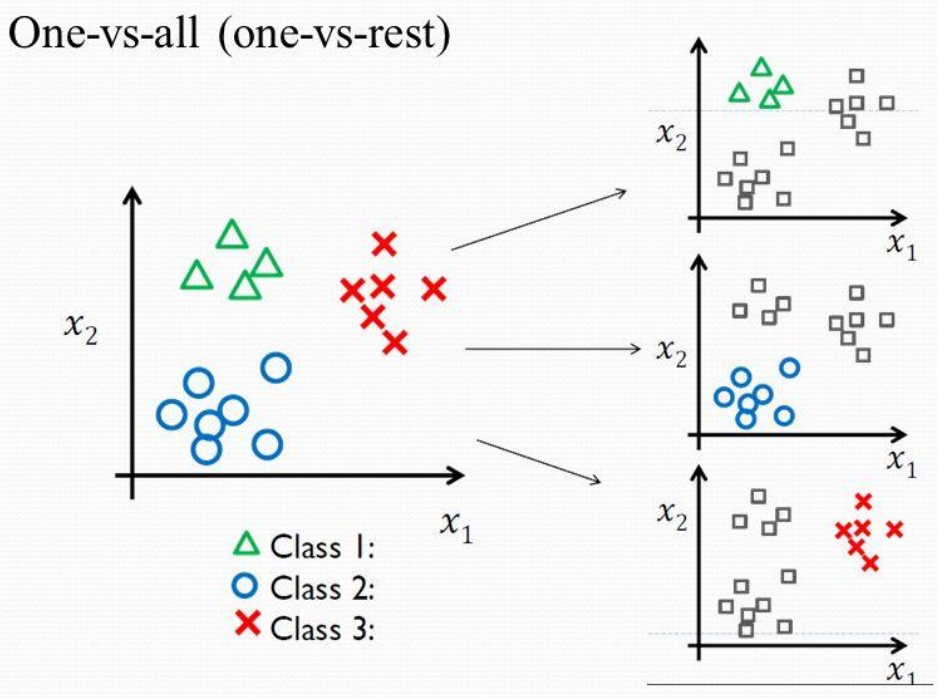
```
classifier.predict([[4,3]])
```

```
array([1])
```



Support Vector Machine

- En un problema con K clases, resolvemos K problemas binarios.
- Cada SVM está entrenada para separar una clase del resto de los patrones.
- Para una nueva instancia x, se corren los K clasificadores y se retorna la clase que tenga una función de decisión con el valor más alto (la clase con mayor confianza).



Support Vector Machine

Conclusiones:

SVM es un algoritmo de aprendizaje supervisado que se propone encontrar el **hiperplano** que mejor separe los datos, tal que se maximice el **margen**.

Hiperparámetros: C y Kernels

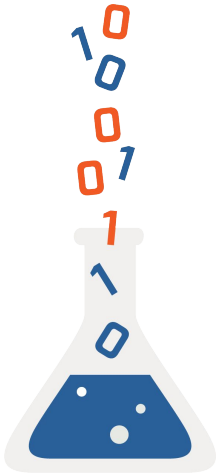
- **Ventajas:**
  - Eficaz en espacios de alta dimensión (incluso cuando son más que el número de instancias!).
  - Eficiente en memoria (sólo los vectores de soporte definen el hiperplano frontera).
  - Los kernels lo hacen muy versátil.
- **Desventajas:**
  - Al usar kernels, hay que tener mucho cuidado de no caer en overfitting.
  - Funciona bien sólo para clasificación.

Support Vector Machine

Enlaces recomendados:

- Support Vector Machines: A Visual Explanation with Sample Python Code
- https://medium.com/datadriveninvestor/support-vector-machines-ae0ff2375479
- https://towardsdatascience.com/kernel-function-6f1d2be6091
- https://medium.com/@sitarzkonrad/interactive-3d-k-means-clustering-in-jupyter-1038470f687e





Lab Time!

¡Muchas gracias!

¡Sigamos trabajando!